

MedAssist AI Frontend & Cloud Milestone Documentation

Prepared By: AKANKSHA MUMMANA

Role

Frontend Developer & Data visualizer

Internship

Infosys Springboard Internship

Project

MedAssist AI – AI Powered Healthcare Prediction System

Table of Contents

Section	Page
1. Project Description	1
2. Technologies Used	2
3. Environment Setup	3
4. Frontend Architecture	4
5. Landing Page Development	5
6. Authentication Module	6
7. Dashboard Module	7
8. Symptom Checker Module	8
9. AI Prediction Module	9
10. Reports Module	10
11. Profile Management	11
12. Docker Containerization	12
13. Challenges & Solutions	13
14. Conclusion	14

1. Project Description

MedAssist AI is an AI-powered healthcare web application designed to assist users in identifying possible diseases based on the symptoms they provide. The application offers a simple, responsive, and user-friendly interface that enables users to enter their symptoms, view AI-generated disease predictions, access previous reports, and manage their personal health profile.

For this milestone, my assigned role was **Frontend Developer & Cloud Engineer**. My primary responsibility was to design and develop the complete frontend interface using **React.js**, ensuring that the application is responsive, modular, and easy to maintain. The frontend was developed with reusable components and client-side routing to provide a smooth user experience.

In addition to frontend development, I configured the project for **Docker-based deployment**, enabling the application to run consistently across different environments. This milestone focused on building the complete user interface while preparing the project for future backend API integration and machine learning model deployment.

Objectives of this Milestone

- Develop a responsive healthcare web interface using React.js.
- Create reusable UI components for better maintainability.
- Implement client-side navigation using React Router.
- Design authentication pages for Login and Registration.
- Develop Dashboard, Symptom Checker, Prediction, Reports, and Profile modules.
- Configure Docker and Docker Compose for containerized deployment.
- Prepare the frontend for backend API integration.

2. Technologies Used

The **MedAssist AI** frontend was developed using modern web technologies to ensure scalability, responsiveness, and maintainability.

Technology	Purpose
React.js	Building reusable frontend components
Vite	Fast development server and optimized build tool
React Router DOM	Client-side navigation between pages
JavaScript (ES6+)	Application logic and functionality
HTML5	Web page structure
CSS3	Styling and responsive layouts
Chart.js	Dashboard analytics and graphical visualization
Local Storage	Persisting user profile information
Docker	Containerizing the frontend application
Docker Compose	Managing Docker containers
Git & GitHub	Version control and collaboration
Visual Studio Code	Source code editor

Development Environment

- Operating System: Windows 11
- IDE: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- Browser Used: Google Chrome
- Container Platform: Docker Desktop

3. Environment Setup

Development Environment:

- Visual Studio Code
- Node.js & npm
- React + Vite
- Docker Desktop
- Git & GitHub

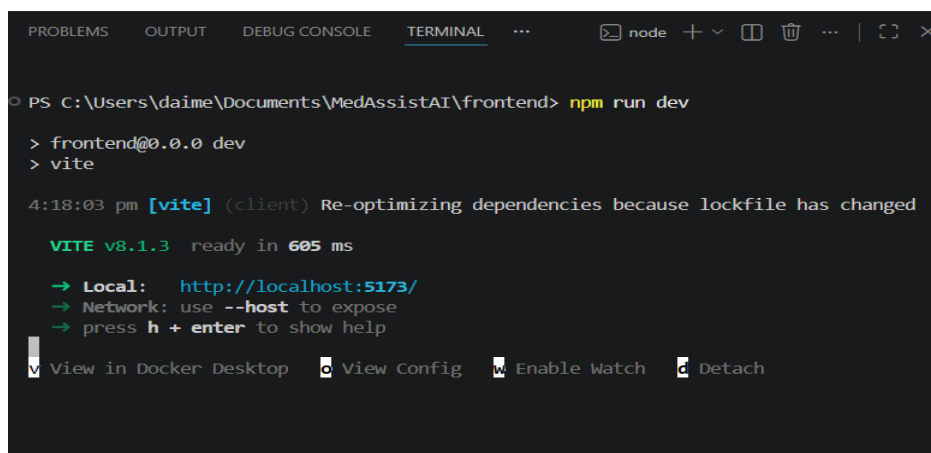
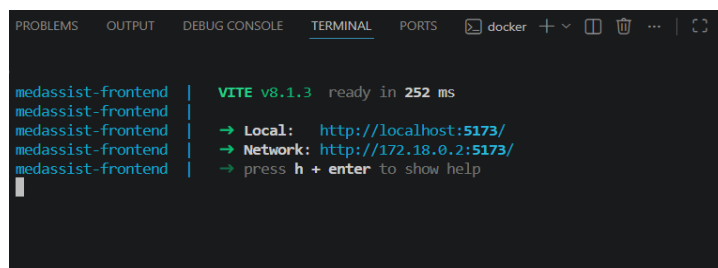
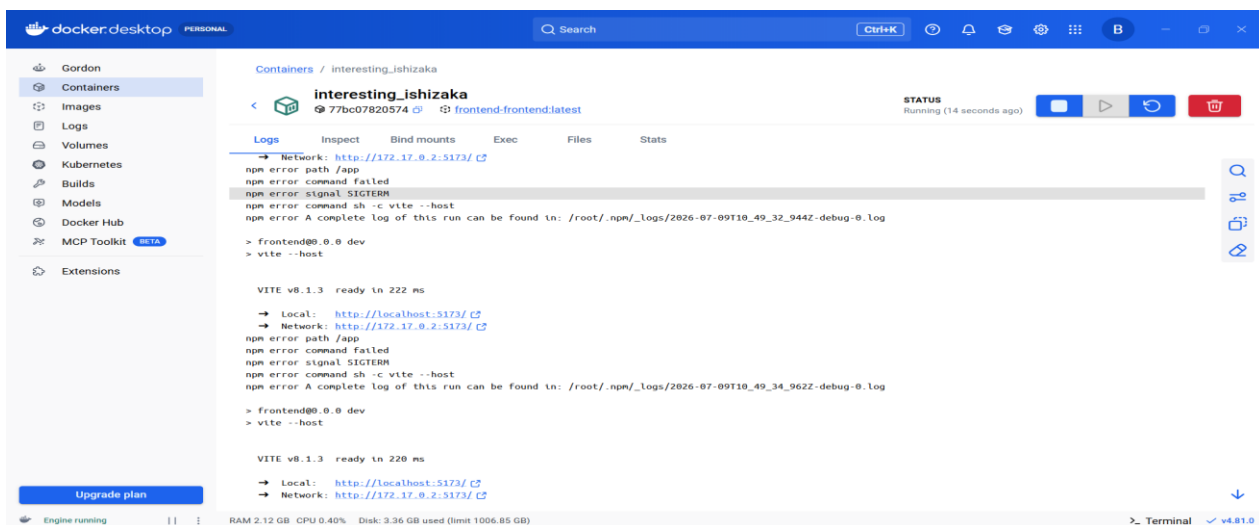
Commands used:

npm install

npm run dev

docker compose up --build

docker compose up

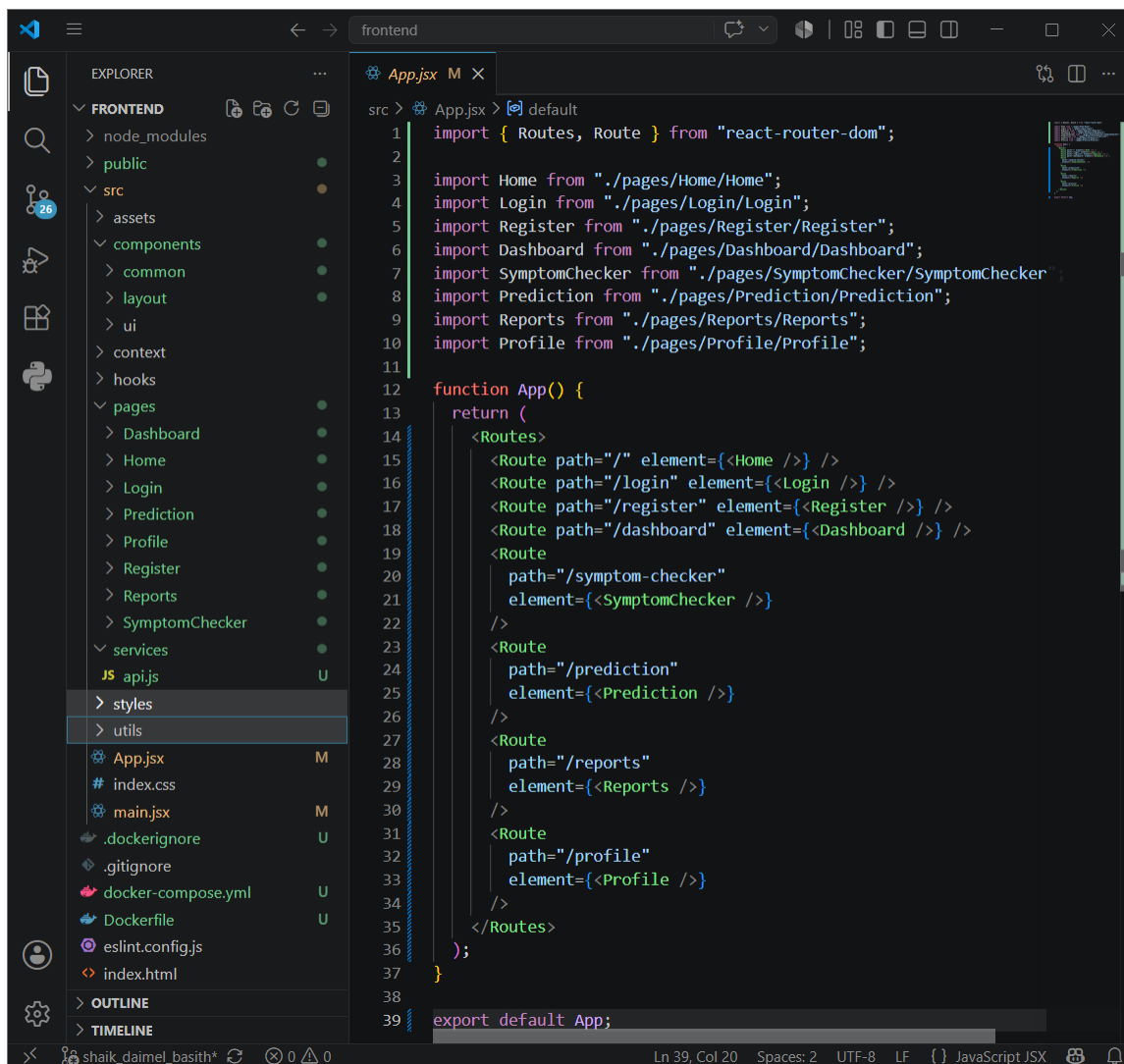


4. Frontend Architecture

Implemented modular folder structure:

- components/
- pages/
- services/
- hooks/
- context/
- styles/

Reusable Layout component with Sidebar Navigation and shared UI.



The screenshot shows the VS Code interface with the 'frontend' folder selected in the Explorer. The folder structure is as follows:

- FRONTEND
 - node_modules
 - public
 - src
 - assets
 - components
 - common
 - layout
 - ui
 - context
 - hooks
 - pages
 - Dashboard
 - Home
 - Login
 - Prediction
 - Profile
 - Register
 - Reports
 - SymptomChecker
 - services
 - api.js
 - styles
 - utils

The 'App.jsx' file is open in the editor, showing the following code:

```
1 import { Routes, Route } from "react-router-dom";
2
3 import Home from "../pages/Home/Home";
4 import Login from "../pages/Login/Login";
5 import Register from "../pages/Register/Register";
6 import Dashboard from "../pages/Dashboard/Dashboard";
7 import SymptomChecker from "../pages/SymptomChecker/SymptomChecker";
8 import Prediction from "../pages/Prediction/Prediction";
9 import Reports from "../pages/Reports/Reports";
10 import Profile from "../pages/Profile/Profile";
11
12 function App() {
13   return (
14     <Routes>
15       <Route path="/" element={<Home />} />
16       <Route path="/login" element={<Login />} />
17       <Route path="/register" element={<Register />} />
18       <Route path="/dashboard" element={<Dashboard />} />
19       <Route
20         path="/symptom-checker"
21         element={<SymptomChecker />}
22       />
23       <Route
24         path="/prediction"
25         element={<Prediction />}
26       />
27       <Route
28         path="/reports"
29         element={<Reports />}
30       />
31       <Route
32         path="/profile"
33         element={<Profile />}
34       />
35     </Routes>
36   );
37 }
38
39 export default App;
```

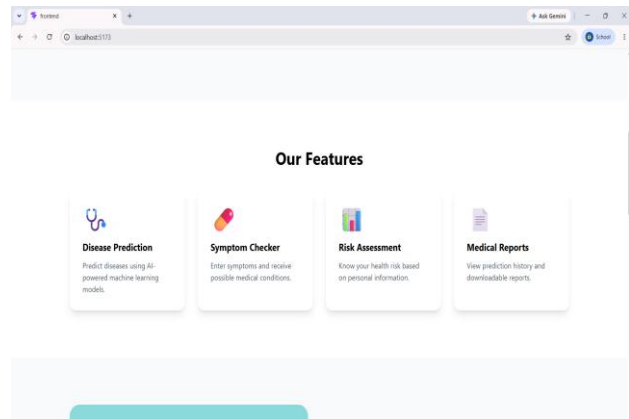
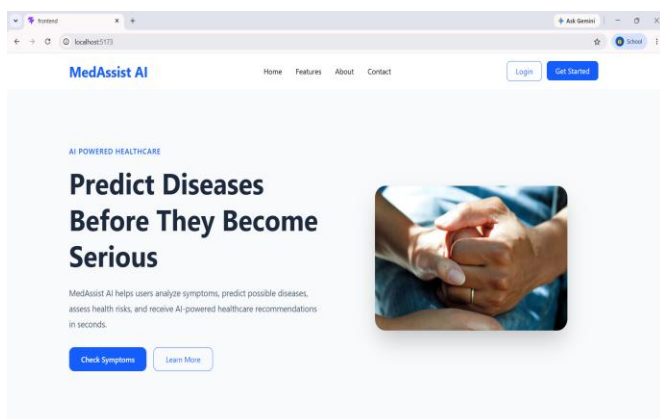
```
frontend
App.jsx M X
src > App.jsx > default
1 import { Routes, Route } from "react-router-dom";
2
3 import Home from "../pages/Home/Home";
4 import Login from "../pages/Login/Login";
5 import Register from "../pages/Register/Register";
6 import Dashboard from "../pages/Dashboard/Dashboard";
7 import SymptomChecker from "../pages/SymptomChecker/SymptomChecker";
8 import Prediction from "../pages/Prediction/Prediction";
9 import Reports from "../pages/Reports/Reports";
10 import Profile from "../pages/Profile/Profile";
11
12 function App() {
13   return (
14     <Routes>
15       <Route path="/" element={<Home />} />
16       <Route path="/login" element={<Login />} />
17       <Route path="/register" element={<Register />} />
18       <Route path="/dashboard" element={<Dashboard />} />
19       <Route
20         path="/symptom-checker"
21         element={<SymptomChecker />}
22       />
23       <Route
24         path="/prediction"
25         element={<Prediction />}
26       />
27       <Route
28         path="/reports"
29         element={<Reports />}
30       />
31       <Route
32         path="/profile"
33         element={<Profile />}
34       />
35     </Routes>
36   );
37 }
38
39 export default App;
```

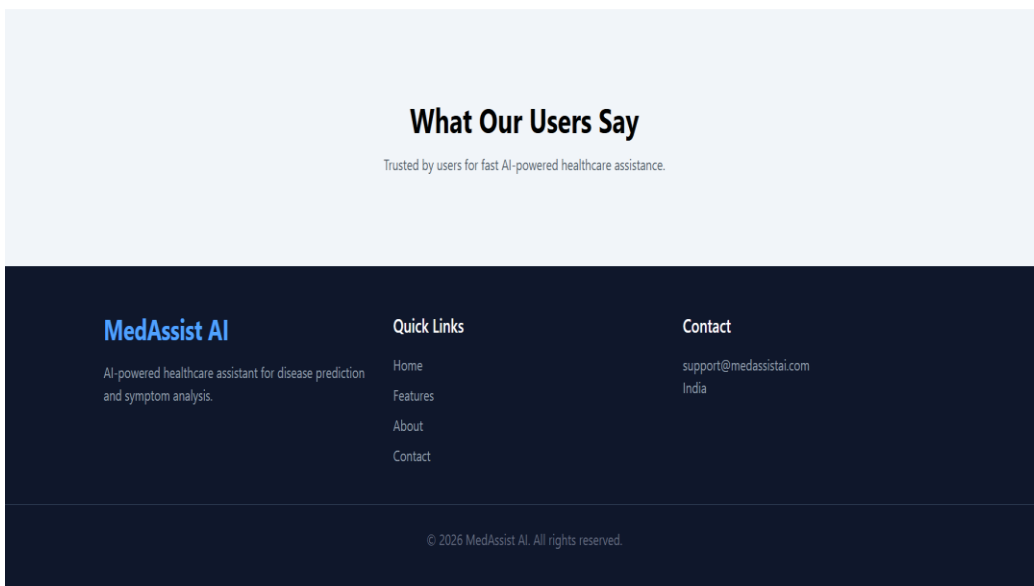
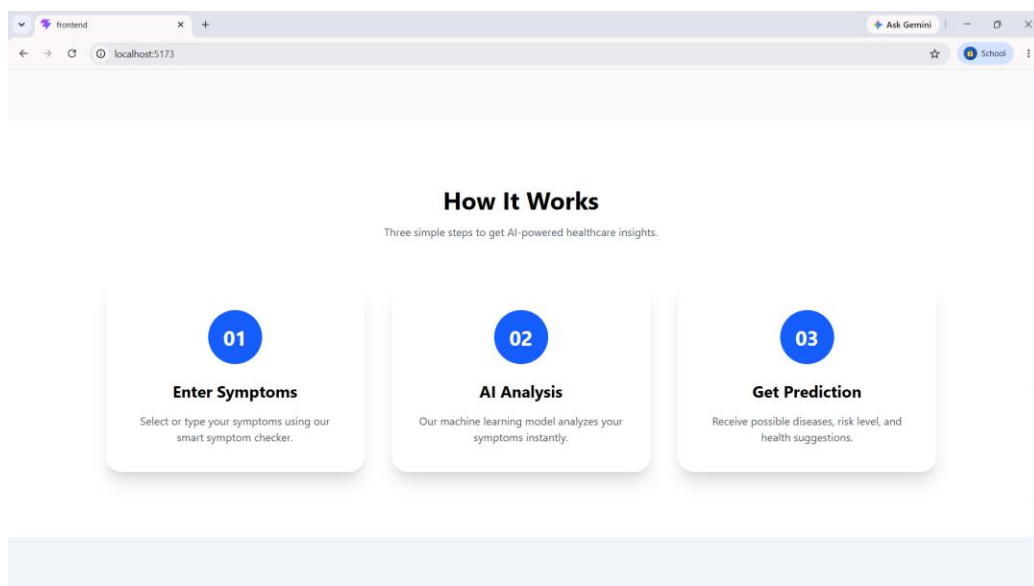
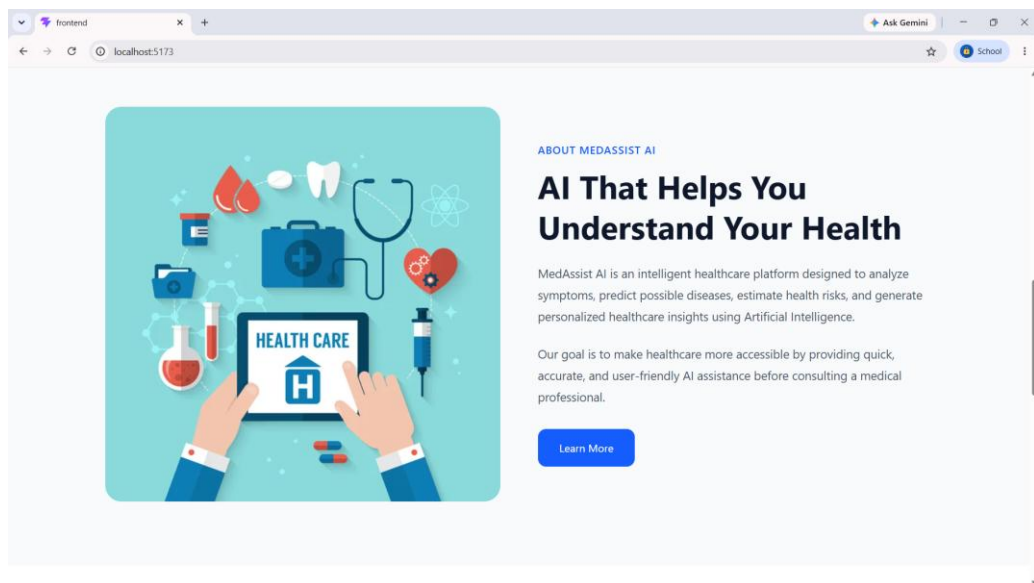
5. Landing Page Development

Implemented Hero, About, Features, How It Works, Testimonials and Footer sections.

Responsive design created using Tailwind CSS.

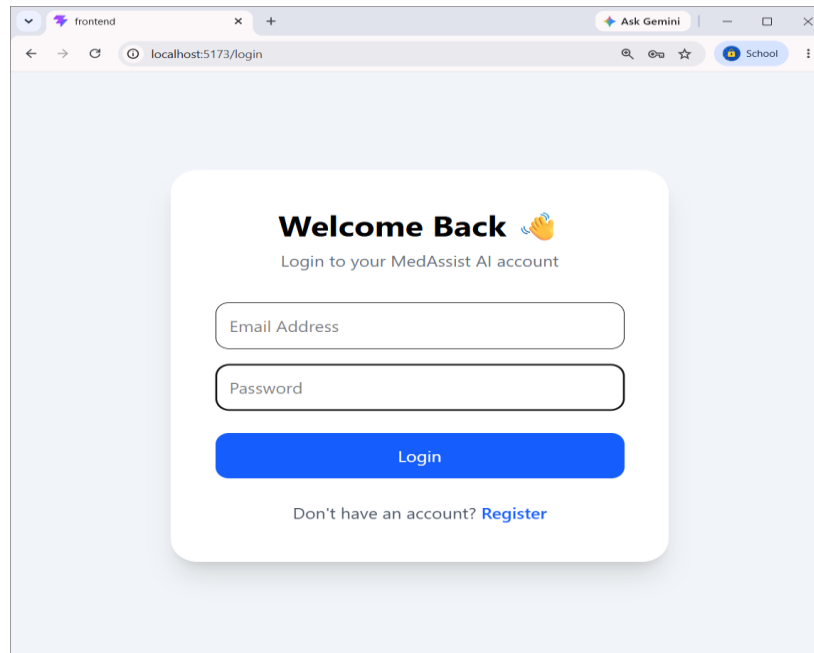
Healthcare illustration integrated from public folder.



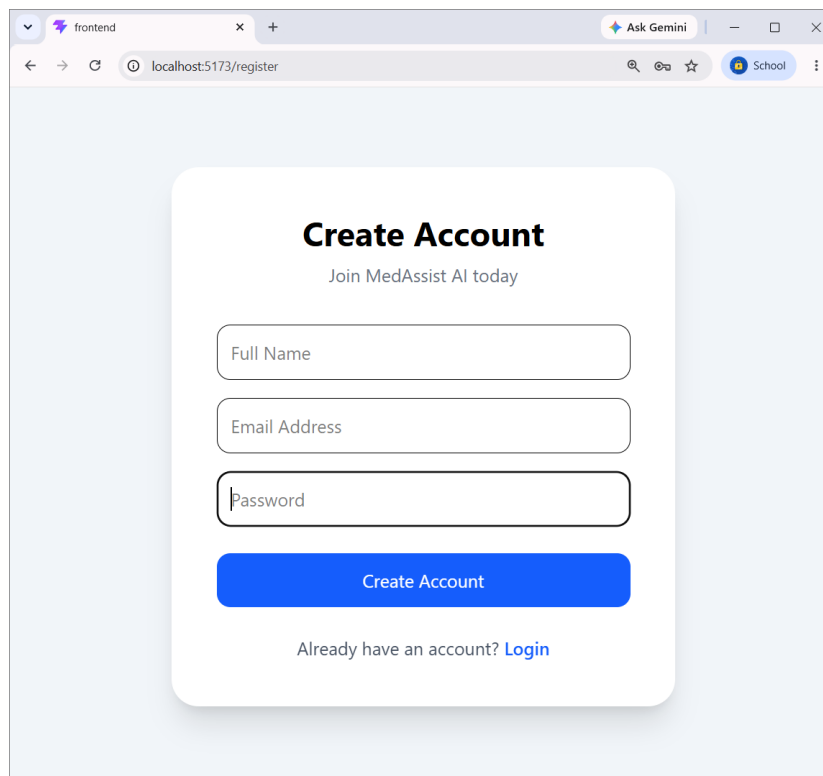


6. Authentication Module

Designed Login and Registration pages with modern responsive UI.
Prepared structure for backend authentication integration.



A screenshot of a web browser displaying the login page. The browser's address bar shows 'localhost:5173/login'. The page features a white card with a light blue background. The card has the heading 'Welcome Back' with a hand icon, followed by the subtext 'Login to your MedAssist AI account'. Below this are two input fields labeled 'Email Address' and 'Password'. A blue 'Login' button is positioned below the fields. At the bottom of the card, it says 'Don't have an account? [Register](#)'.

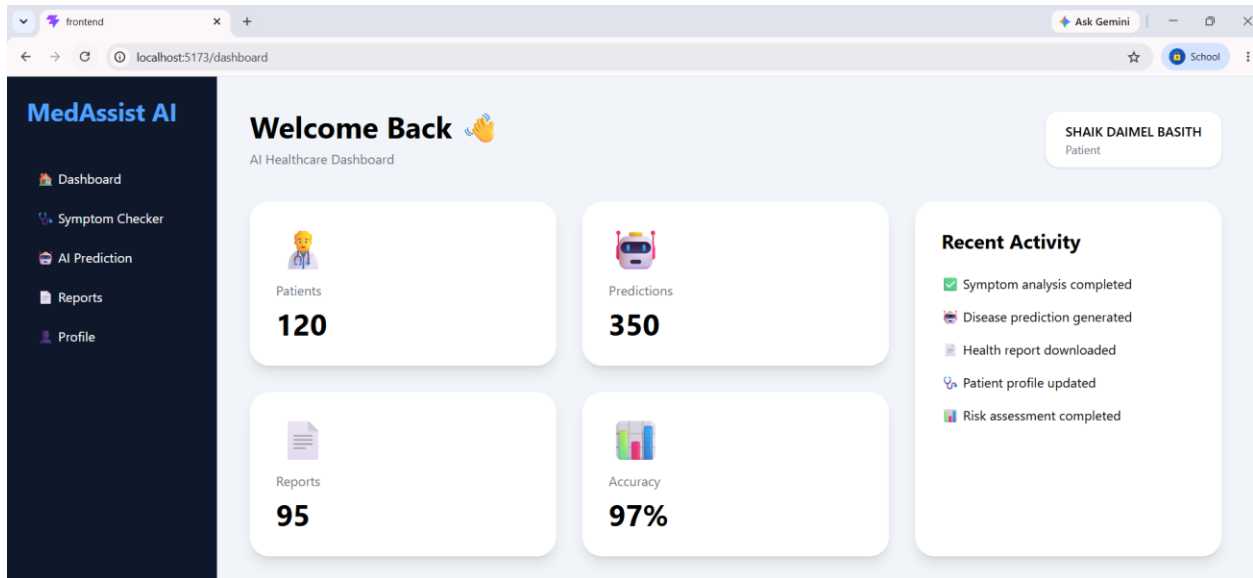


A screenshot of a web browser displaying the registration page. The browser's address bar shows 'localhost:5173/register'. The page features a white card with a light blue background. The card has the heading 'Create Account' with the subtext 'Join MedAssist AI today'. Below this are three input fields labeled 'Full Name', 'Email Address', and 'Password'. A blue 'Create Account' button is positioned below the fields. At the bottom of the card, it says 'Already have an account? [Login](#)'.

7. Dashboard Module

Developed Dashboard with:

- Sidebar Navigation
- Statistic Cards
- Recent Activity Panel
- Disease Analytics using Chart.js Bar Chart.



8. Symptom Checker Module

Created symptom selection interface with:

- Age
- Gender
- Height
- Weight
- Multiple symptom selection

Navigation implemented to Prediction page.

The screenshot shows a web browser window with the URL `localhost:5173/symptom-checker`. The page features a dark blue sidebar on the left with the logo "MedAssist AI" and navigation links: "Dashboard", "Symptom Checker", "AI Prediction", "Reports", and "Profile". The main content area is titled "Symptom Checker" with the instruction "Select your symptoms and let AI analyze them." Below this, there are input fields for "Age", "Gender" (a dropdown menu showing "Male"), "Height (cm)", and "Weight (kg)". There are also ten checkboxes for symptoms: "Fever", "Cough", "Headache", "Fatigue", "Chest Pain", "Shortness of Breath", "Vomiting", "Sore Throat", "Body Pain", and "Nausea". At the bottom of the form is a large blue button labeled "Analyze Symptoms".

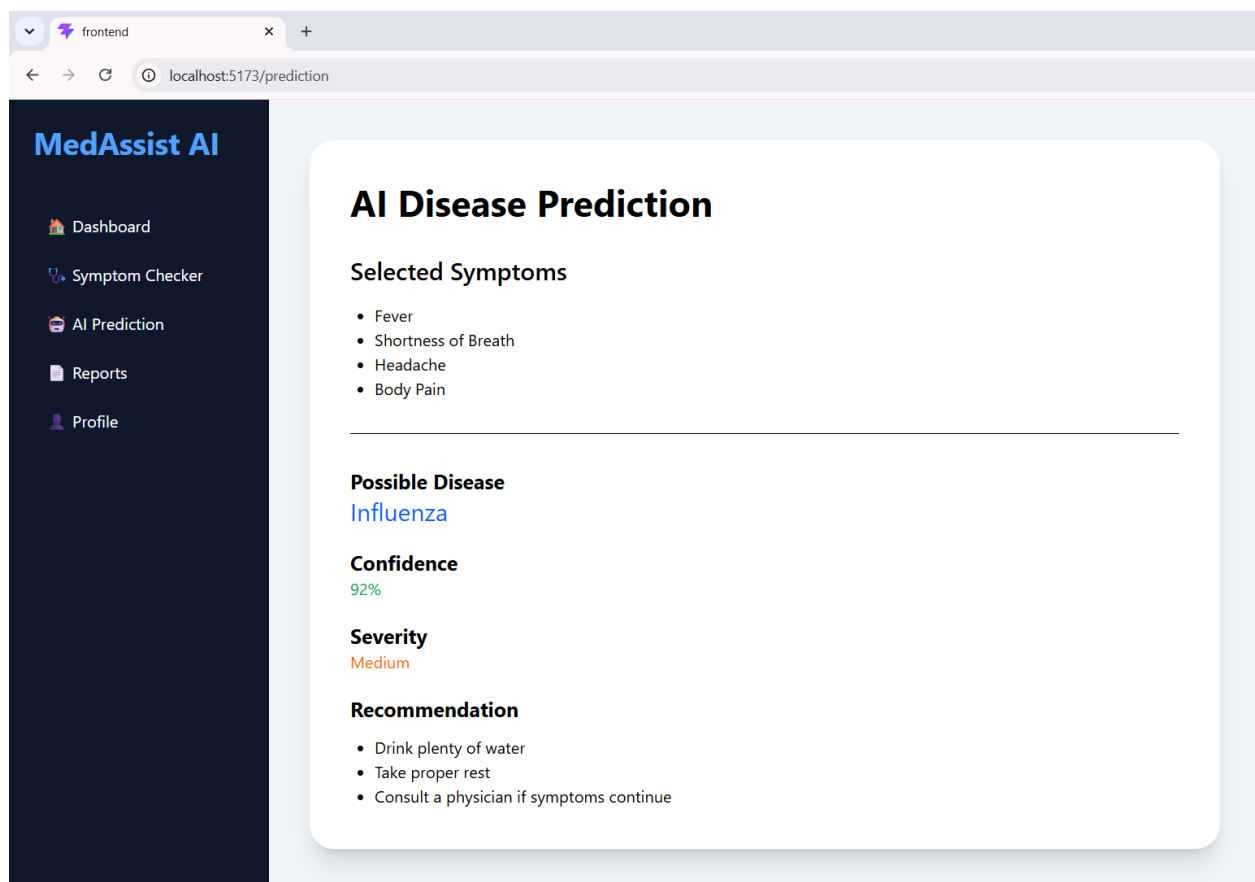
This screenshot shows the same "Symptom Checker" interface, but with data entered. The "Age" field contains "18", "Height (cm)" contains "168", and "Weight (kg)" contains "60". The "Gender" dropdown is still set to "Male". Under the symptoms, the checkboxes for "Fever", "Headache", "Body Pain", and "Shortness of Breath" are now checked. The "Analyze Symptoms" button remains at the bottom.

9. AI Prediction Module

Displays:

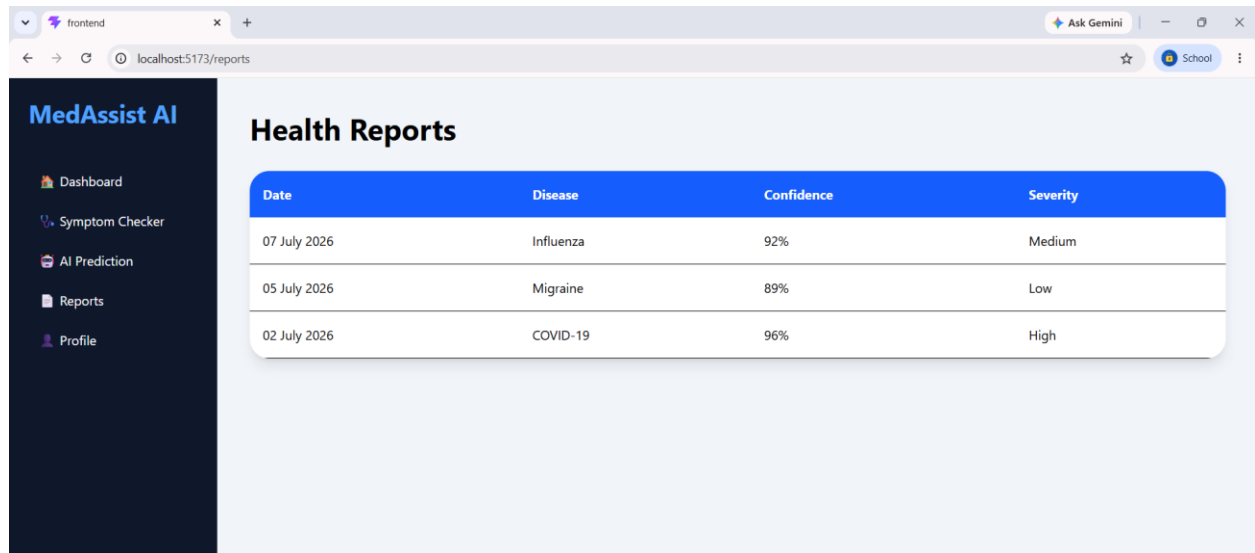
- Selected Symptoms
- Predicted Disease
- Confidence Score
- Severity
- AI Recommendation

Currently connected using frontend mock data.



10. Reports Module

Created Reports page UI for displaying patient reports and future downloadable reports. **Working of Reports Module**

The screenshot shows a web browser window with the URL 'localhost:5173/reports'. The browser's address bar also shows 'frontend'. The page has a dark blue sidebar on the left with the 'MedAssist AI' logo and navigation links: 'Dashboard', 'Symptom Checker', 'AI Prediction', 'Reports' (highlighted), and 'Profile'. The main content area is titled 'Health Reports' and contains a table with four columns: 'Date', 'Disease', 'Confidence', and 'Severity'. The table lists three reports: one for Influenza on 07 July 2026 with 92% confidence and medium severity, one for Migraine on 05 July 2026 with 89% confidence and low severity, and one for COVID-19 on 02 July 2026 with 96% confidence and high severity.

Date	Disease	Confidence	Severity
07 July 2026	Influenza	92%	Medium
05 July 2026	Migraine	89%	Low
02 July 2026	COVID-19	96%	High

The **Reports Module** provides users with a centralized view of their previously generated AI health prediction reports. Each report displays important information such as the prediction date, detected disease, confidence score, and severity level. This enables users to track their health assessment history and review previous prediction results whenever required.

The interface is designed using a responsive table layout to ensure clear visualization of report details across different screen sizes. The module is structured in a way that allows seamless integration with the backend, where future versions will retrieve report data dynamically from a database and support downloading reports in PDF format.

Key Features:

- Displays prediction history in a tabular format.
- Shows prediction date, disease name, confidence percentage, and severity level.
- User-friendly and responsive interface.
- Prepared for backend API integration.
- Designed to support future report download functionality.

11. Profile Management

Implemented editable profile.

Implemented profile image upload.

Stored profile information and image using browser Local Storage so data persists after refresh.

The screenshot shows a web browser window with the URL `localhost:5173/profile`. The page features a dark blue sidebar on the left with the logo "MedAssist AI" and a menu with items: Dashboard, Symptom Checker, AI Prediction, Reports, and Profile. The main content area is a light blue card with a profile picture of a man, the name "SHAIK DAIMEL BASITH", and the title "Patient". Below this, there are four input fields: "Age" (20), "Gender" (Male), "Height" (168 cm), and "Weight" (59 kg). A blue "Edit Profile" button is at the bottom of the card.

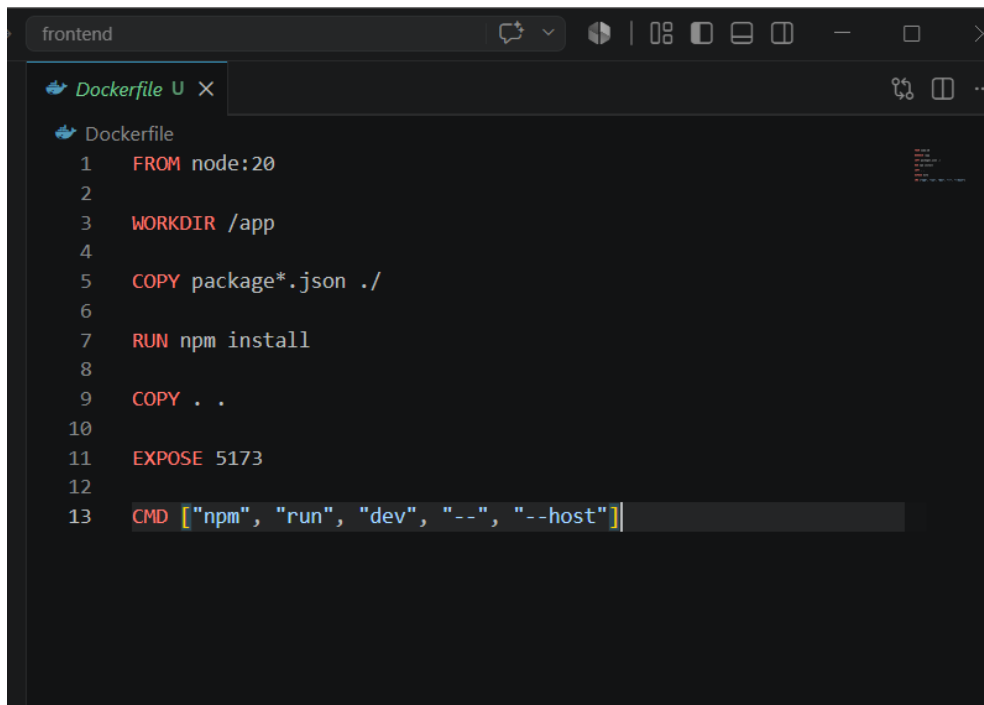
The screenshot shows the same web browser window, but the profile information has been updated. The "Age" field now contains "18" and the "Weight" field now contains "60 kg". The "Height" field remains "168 cm" and the "Gender" field remains "Male". The blue button at the bottom is now labeled "Save Profile". A white toast notification is displayed at the top center, stating "localhost:5173 says Profile Updated Successfully" with a green checkmark and an "OK" button.

12. Docker Containerization

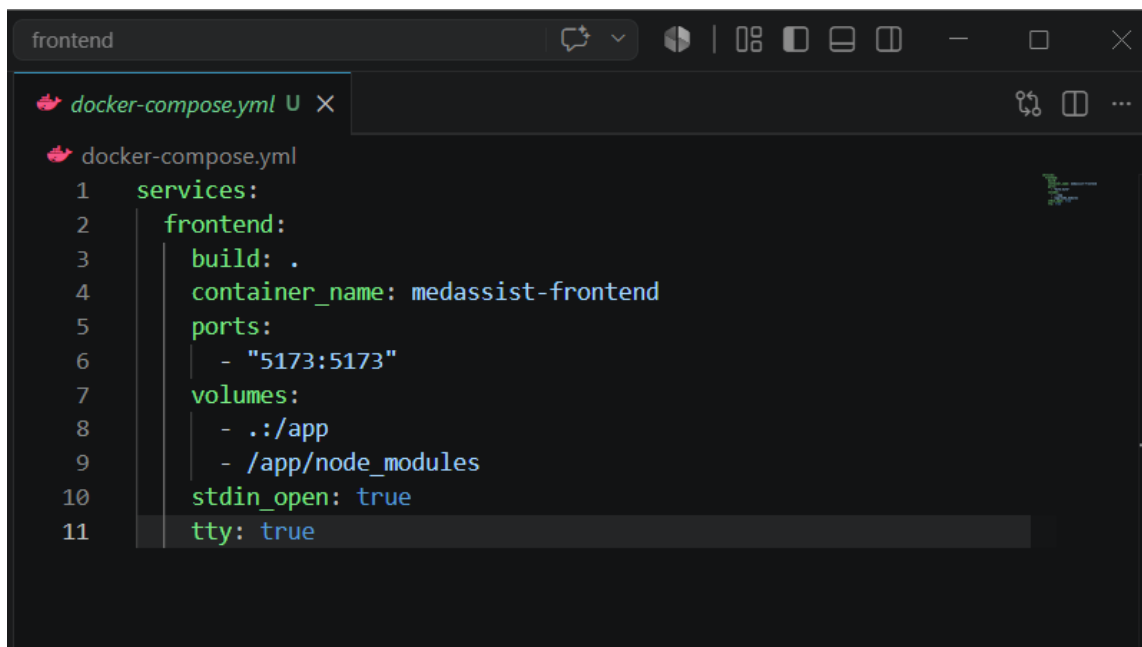
Created:

- Dockerfile
- docker-compose.yml
- .dockerignore

Successfully executed frontend inside Docker container using Docker Desktop.

A screenshot of a code editor window titled 'frontend'. The editor shows a Dockerfile with the following content:

```
Dockerfile
1 FROM node:20
2
3 WORKDIR /app
4
5 COPY package*.json ./
6
7 RUN npm install
8
9 COPY . .
10
11 EXPOSE 5173
12
13 CMD ["npm", "run", "dev", "--", "--host"]
```

A screenshot of a code editor window titled 'frontend'. The editor shows a docker-compose.yml file with the following content:

```
docker-compose.yml
1 services:
2   frontend:
3     build: .
4     container_name: medassist-frontend
5     ports:
6       - "5173:5173"
7     volumes:
8       - ./app
9       - /app/node_modules
10    stdin_open: true
11    tty: true
```

13. Challenges & Solutions

Challenges Faced

During the development of the MedAssist AI Frontend and Cloud module, several technical challenges were encountered. Configuring React Router for smooth navigation between multiple pages required careful route management and component organization. Integrating the common sidebar layout across dashboard pages while maintaining responsive design also required additional styling adjustments.

Another challenge was implementing persistent user profile information. Initially, profile details were lost after refreshing the page. This issue was resolved by storing user information using the browser's Local Storage. Static asset loading inside the React application also required modifications to the folder structure and public asset references.

While containerizing the application, Docker build errors occurred due to dependency installation and configuration issues. These were resolved by updating the Dockerfile, configuring docker-compose correctly, and verifying the application inside Docker Desktop.

Solutions Implemented

- Configured React Router for seamless page navigation.
- Developed reusable layout components for consistency.
- Implemented Local Storage for persistent profile data.
- Corrected image asset paths using the public directory.
- Optimized Dockerfile and docker-compose configuration.
- Successfully tested the frontend application inside Docker containers.
- Verified responsive design across desktop and mobile screen sizes.

14. Conclusion

The Frontend Development and Cloud Engineering milestone for the **MedAssist AI** project has been completed successfully. A complete and responsive frontend application was developed using React.js and Vite, providing an intuitive user interface for healthcare prediction. The application includes a landing page, user authentication screens, dashboard, symptom checker, AI prediction module, health reports page, and profile management functionality.

Cloud deployment support was implemented using Docker and Docker Compose, enabling the frontend to run consistently in a containerized environment. The application was tested successfully in both local and Docker environments, ensuring portability and ease of deployment.

This milestone establishes a strong foundation for the next phase of the project, where backend APIs, machine learning models, and database connectivity will be integrated to provide real-time disease prediction and personalized healthcare recommendations. The completed frontend architecture is modular, scalable, and ready for future enhancements.

Outcome

- Successfully developed a responsive React-based frontend.
- Implemented modular architecture with reusable components.
- Configured client-side routing using React Router.
- Designed user-friendly healthcare interfaces.
- Containerized the frontend using Docker.
- Prepared the application for backend integration and deployment.